



BILDUNG. FREUDE INKLUSIVE.

Webdesign im IT-L@B

Einführung in HTML und CSS
Responsive Webseiten

Überarbeitet: 29.10.2024

von Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

Arbeitsauftrag:

1. **Lesen** Sie das Skript **vollständig**!
2. **Programmieren** Sie die vorgestellten Beispiele – auch in eigenen Varianten – nach. Vermeiden Sie es, fremde Programmskripte zu kopieren!
3. **Bearbeiten** Sie den am Ende der Lektion gestellten Arbeitsauftrag!
4. **Testen** Sie die Webseite auf verschiedenen Webbrowsern!

**Die Code-Beispiele sind als Grafik in dieses Dokument eingefügt.
D.h.: Copy&Paste ist keine mögliche Lösung!**

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen!!!!

Kalkulierte Bearbeitungszeit

Die Aufgabe ist so gestaltet, dass Sie einschließlich der Lektüre dieser Lektion, dem Ausprobieren der Beispiele und eventuell eigene Recherchen im Internet maximal **4 Stunden Arbeitszeit** benötigen sollten. Wenn Sie Probleme (z.B. Schwierigkeiten mit dem Verständnis der deutschen Sprache) haben sollten, wenden Sie sich bitte an Ihre Trainerin oder Ihren Trainer.

Für die Abgabe der Aufgaben gelten folgende Regeln:

Offizielle Lehrlinge des IT-L@B:

Lehrlinge des IT-@B verfügen über einen Zugang zum Lernsystem *Moodle*. Rufen Sie den folgenden Kurs auf:

20E 55 - User Experience - IT-L@B Villach

Geben Sie die Arbeit im dafür vorgesehenen Abgabefeld der Aufgabe fristgerecht ab.

Praktikantinnen und Praktikanten:

Praktikantinnen und Praktikanten haben noch keinen Zugang zum Moodle-System. Sie geben die Aufgabe bitte in Ihrem **persönlichen Ordner** auf dem NAS („\\itlabvi\\public“) ab. Richten Sie dafür bitte – falls dies noch nicht geschehen ist – ein Unterverzeichnis „web“ ein.

Inhalt

Arbeitsauftrag:	2
Kalkulierte Bearbeitungszeit	2
Für die Abgabe der Aufgaben gelten folgende Regeln:	2
Offizielle Lehrlinge des IT-L@B:.....	2
Praktikantinnen und Praktikanten:	2
Begriff: responsiv	4
@media-Query	4
@media screen, @media print und @media all	4
Beispiel für ein CSS-Format zum Ausdruck auf Papier (A4)	5
Beispiel für ein CSS-Format zur Darstellung auf dem Bildschirm.....	5
Experimentierauftrag:	5
Konkretisierung der @media-Queries mit Medienmerkmalen	6
@media-Queries mit kombinierten	8
@media-Query mit dem Medienmerkmal „width“ bzw. „min-/max-width“	8
@media-Query mit dem Medienmerkmal „height“ bzw. „min-/max-height“	9
@media-Query mit dem Medienmerkmal „orientation“	9
@media-Query mit dem Medienmerkmal „aspect-ratio“	11
Bereiche mit @media-Queries:	13
@container	13
Container-Typen.....	14
Container-Struktur und Container-Name	14
@container - Größenabfragen	14
@container – Stil-Abfragen	17
Aufgaben	17

Begriff: responsiv

Eine wichtige Eigenschaft, die von modernen Webseiten erwartet wird, ist neben der **Barrierefreiheit** und einer **schnellen Performance** der Seite auch die Anpassung an die verschiedenen Endgerätetypen, mit denen die Webseite betrachtet werden kann. Man spricht hier von einem **responsiven Design**.

Im Modul 7 – Bilder und Multimedia – wurden bereits erste Ansätze eines responsiven Designs vorgestellt: Mithilfe der Elemente `<picture>` und `<source>` wurde gezeigt, wie eine bildliche Darstellung mit verschiedenen Bildquellen (angepasste Dateigrößen) für Viewports mit verschiedenen Bildschirmbreiten optimiert ausgewählt werden kann. Damit wurde es möglich, das Datenvolumen für die Übertragung einer Webseite an das verwendete Gerät – und einem möglicherweise dazu passenden Übertragungsmedium – anzupassen.

Bei der in diesem Modul beschriebenen responsiven Anpassung einer Webseite wird es um die Struktur der Seite gehen. Die Ausführungen betreffen ausschließlich CSS-Formate. Darüber hinaus stellt dieses Modul die Grundlage für die nachfolgenden Module dar, bei denen es um den Einsatz der Flex- und Grid-Technologie gehen wird.

@media-Query

Medienabfragen werden in CSS verwendet, um Formate für bestimmte Ausgabemedien zu programmieren. Der Browser prüft beispielsweise für welches Ausgabemedium die Webseite aktuell aufzubereiten ist. Dabei wird zwischen dem Bildschirm (screen) und einem Drucker (print) unterschieden. Auch können beide Ausgabemedien in einer Medienabfrage berücksichtigt werden.

@media screen, @media print und @media all

In der Regel werden Webseiten auf einem Bildschirm ausgegeben. Allerdings kann es natürlich auch passieren, dass eine Seite zum Ausdruck aufzubereiten ist. Das erfordert spezielle Formate. Jede Person, die einmal versucht hat, eine Webseite mit längerem Inhalt auszudrucken, kennt das Problem. Es werden Randbereiche (z.B.: Werbung) gedruckt oder auch Bilder beim Seitenumbruch in Teilen zu Papier gebracht.

Grob ausgedrückt lässt sich sagen, dass es mithilfe von @media-Queries möglich ist, die für das gewünschte Ausgabemedium erforderlichen Formate gezielt einzustellen. Ein Beispiel:

Für die Ausgabe auf einem Bildschirm werden bevorzugt Maßeinheiten in „Bildpunkten“ (px) verwendet. Diese Maßeinheiten sind für einen Ausdruck auf dem Papier nicht brauchbar. Hier werden konkrete Maßeinheiten in Zentimeter oder Millimeter (bei internationalen Formaten auch inch etc.) benötigt. Diese beziehen sich auf die Abmessungen einer Druckseite.

Auch Farben werden an das Ausgabemedium angepasst. So ist es völlig sinnlos, eine weiße Schrift auf ein weißes Blatt Papier zu drucken. Der Ausdruck eines dunklen Hintergrundes – dieser würde die weiße Schrift natürlich möglich machen – stellt allerdings eine eklatante Verschwendung von Toner bzw. Tinte dar und sollte vermieden werden. Wird die gleiche Seite allerdings auf dem Bildschirm präsentiert, so sind die Voraussetzungen für die Farbgebung anders. Dunkle Hintergründe schonen die Augen und sind auch aus Gründen des Designs oft eine gute Wahl.

Sollen @media-Queries für alle denkbaren Medien definiert werden, passiert dies mit dem Parameter „all“.

Hinweis: Im folgenden Beispiel werden lediglich die Medien bezeichnet. Bestimmte Einschränkungen wie Bildschirmbreite etc. sind hier noch nicht relevant.

Beispiel für ein CSS-Format zum Ausdruck auf Papier (A4)

```
@media print {
body {
width: 17cm;
height: 25,4cm
color: black;
}
}
```

Prüfen Sie nun das Ergebnis der Formatierung: „Drucken“ Sie die Seite mit beliebigem Inhalt (im Beispiel wurde nur ein <h1>-Element verwendet) als PDF in eine Datei.

Beispiel für ein CSS-Format zur Darstellung auf dem Bildschirm

```
@media screen {
body {
width: 1200px;
height: 500px;
color: white;
background-color: blue;
}
}
```

Das folgende Bild zeigt auf der linken Seite die Wirkung des @media-Queries für den Screen (Bildschirm). Rechts daneben ist das Ergebnis der oben beschriebenen Formatierung für den Ausdruck zu sehen.

Experimentierauftrag:

Entfernen Sie aus dem ersten Beispiel (@media print) die Zeile **color: black;**. „Drucken“ Sie erneut die Webseite (Druckertreiber: „Print to PDF“) in eine PDF-Datei. Was passiert? Prüfen Sie vor allem die Farbgebungen für die Schrift und den Hintergrund.

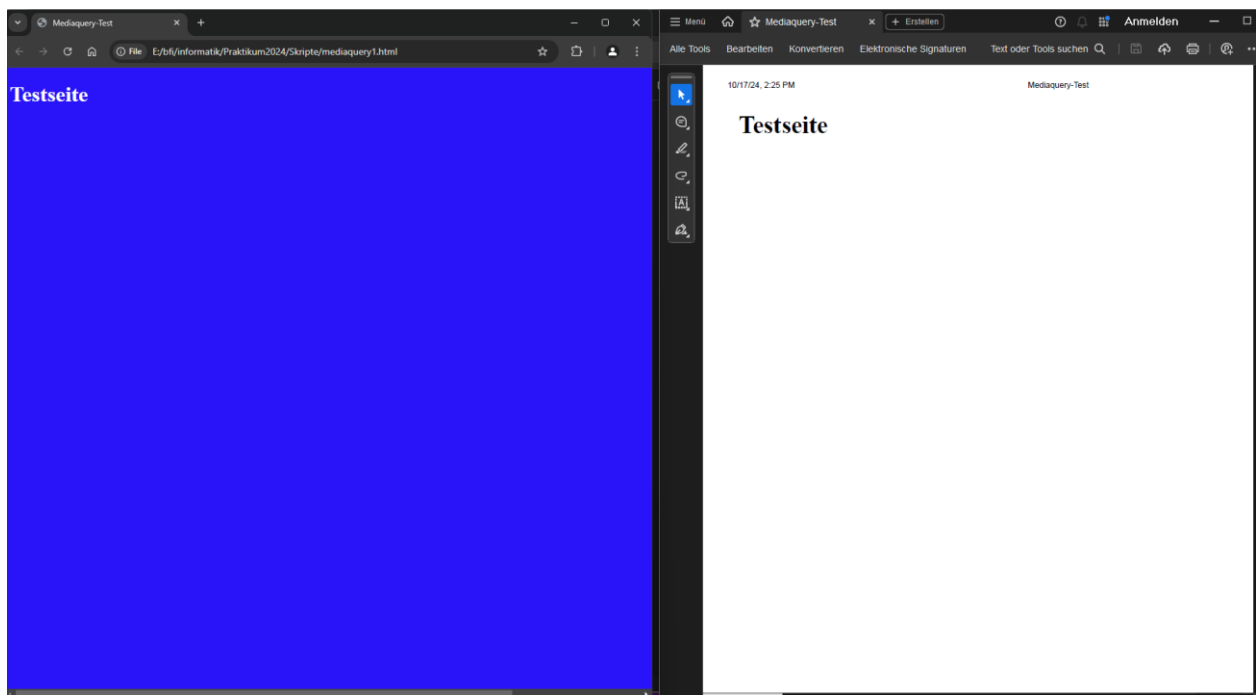


Abbildung 1: Links: Darstellung auf dem Bildschirm, rechts die gleiche Seite als Ausdruck.

Konkretisierung der @media-Queries mit Medienmerkmalen

Neben der Art des Ausgabemedium ist es sinnvoll, die Eigenschaften der Ausgabe weiter zu konkretisieren. Es gibt verschiedene Eigenschaften, die in @media-Queries ausgewertet werden können. Dieses Modul wird sich auf eine Auswahl beschränken:

- width
- height
- orientation
- aspect-ratio

Ein Beispiel hierfür – und dieses ist für die weitere Programmierung im Rahmen dieser Ausbildung sehr wichtig – ist die Betrachtung der verfügbaren Bildschirmbreite bzw. dessen Höhe.

Bei der Erarbeitung der Screendesigns wurde möglicherweise deutlich, dass die Anordnung mehrerer Elemente mit einer bestimmten Breite (in px) erst ab einer bestimmten Mindest-Bildschirmbreite ohne Umbruch nebeneinander möglich ist. Für Bildschirmbreiten unterhalb dieser Mindestbreite müssen also andere Layout-Konzepte erdacht werden.

Der folgende HTML-Code (Auszug) zeigt eine Menüleiste mit sechs Schaltflächen:

```
<body>
<nav>
<ul>
  <li><a href="#">Button 1</a></li>
  <li><a href="#">Button 2</a></li>
  <li><a href="#">Button 3</a></li>
  <li><a href="#">Button 4</a></li>
  <li><a href="#">Button 5</a></li>
  <li><a href="#">Button 6</a></li>
</ul>
</nav>
...
</body>
```



Abbildung 2: Im idealen Fall werden die hier programmierten sechs Schaltflächen nebeneinander auf dem Bildschirm dargestellt.

Eine solche „ungeordnete Liste“ ist bereits aus den ersten Modulen bekannt. Die Hyperlinks sind hier nur symbolisch durch das Zeichen # dargestellt. Aus der Liste wird mithilfe des folgenden CSS-Codes eine Menüleiste (waagrechte Ausrichtung) mit sechs Schaltflächen:

```
nav ul {  
  padding-left: 0px;  
  list-style: none;  
}  
  
nav li {  
  display: inline-block;  
  margin: 5px;  
}  
  
nav a {  
  display: block;  
  width: 150px;  
  height: 25px;  
  padding-top: 7px;  
  background-color: lightblue;  
  color: blue;  
  text-align: center;  
  border-radius: 25px 0px 25px 0px;  
}
```

Jedes `<a>`-Element hat eine Breite von 150px. Hinzu kommen zwischen den Elementen sowie links neben dem ersten Element und rechts neben dem letzten Element ein Außenabstand von jeweils 5px. Die Menüleiste hat also eine Breite von

$$6 \cdot 150px + 7 \cdot 5px = 935px$$

Der hier verwendete Chrome-Browser stellt die Menüleiste nur dann korrekt dar, wenn die Viewport-Breite mindestens einen Bildpunkt größer ist. Wünschenswert ist eine Darstellung, wie sie Abbildung 2 zeigt.

In Abbildung 3 wurde die Viewport-Breite auf 935px eingestellt. Deswegen erscheint die sechste Schaltfläche in einer neuen Zeile.



Abbildung 3: Hier beträgt die Viewport-Breite 935px. Die Schaltflächen passen nun nicht mehr nebeneinander auf den Bildschirm. Es erfolgt ein Umbruch, wodurch die sechste Schaltfläche in einer neuen Zeile dargestellt wird. Dies ist nicht nur rein optisch unschön, sondern bei Menüs mit mehreren Ebenen auch funktional schwierig.

Hinweis: Unverkennbar wird spätestens hier die Bedeutung eines gut ausgearbeiteten Screendesigns klar: Die erforderlichen Werte der Parameter in den Media-Queries können direkt übernommen werden.

@media-Queries mit kombinierten Medienmerkmalen

Man kann durch Kombinationen verschiedener Medienmerkmale den Eigenschaften des Ausgabegerätes sehr nahekommen. Dies ist heutzutage jedoch nicht mehr ganz einfach, denn Tablets und Smartphones haben keinesfalls mehr eingeschränkte Eigenschaften hinsichtlich der Bildschirmauflösung. Man kann allerdings verschiedene der nachfolgend beschriebenen Medieneigenschaften kombinieren. Hierzu werden bekannte logische Verknüpfungsoperatoren verwendet:

- and
- not

Tatsächlich ist keine oder-Verknüpfung für CSS-@media-Queries vorgesehen! Man kann allerdings – entsprechend der gängigen CSS-Syntax – mehrere @media-Queries durch Kommata getrennt in einem Selektor schreiben.

Im Folgenden soll der Fokus auf Kombinationen mit „and“ liegen.

@media-Query mit dem Medienmerkmal „width“ bzw. „min-/max-width“

Im Beispiel wurde eine Navigationsleiste mit einer gesamten Breite von 935px gezeigt. Um diese Menüleiste tatsächlich ohne einen Umbruch in eine zweite Zeile darzustellen, muss die Viewport-Breite also mindestens 936px betragen. Zudem soll die Darstellung mit den gezeigten Formaten nur auf einem Monitor erfolgen.

Wir kombinieren also die Anforderungen für das geeignete Format „Darstellung auf einem Bildschirm (Screen) mit einer Mindestbreite von 936px in einer @media-Query:

```
@media screen and (min-width: 936px) {...
```

Es stellt sich nun die Frage, wie die Menüleiste bei schmalere Viewports darzustellen ist. Man könnte zum Beispiel die Schaltflächen verkleinern oder man könnte das Menü vertikal ausrichten.

Beides setzt wieder einen individuellen Regelblock voraus, der nur dann wirksam werden soll, wenn die Bildschirmbreite kleiner oder gleich 935px (Achtung: Nur im hier gezeigten Beispiel!) ist.

Die hierzu verwendete Medienabfrage lautet:

```
@media screen and (max-width: 935px) {...
```

In den Block mit den geschweiften Klammern schreibt man lediglich die CSS-Regeln hinein, die für den betreffenden Fall verändert werden müssen. Man kopiert also NICHT den kompletten allgemein gültigen Code in den neuen Regelblock. Dies würde den CSS-Code nur unnötig aufblähen.

Damit das Menü nicht mehr horizontal, sondern nunmehr vertikal dargestellt wird, soll zuerst einmal der Code auf Seite 7 betrachtet werden. Hier ist ein CSS-Regelblock für alle -Elemente innerhalb eines <nav>-Containers zu sehen:

```
nav li {  
    display: inline-block;  
    margin: 5px;  
  
}
```

-Elemente sind per se Block-Elemente. Jeder neue -Container wird also immer in einer neuen Zeile dargestellt. Die Regel „display: inline-block;“ bewirkt jedoch, dass der Container nach innen seine ursprüngliche Eigenschaft behalten kann. Nach außen wird dieses Element jedoch als inline-Element verstanden, was bewirkt, dass die Schaltflächen nebeneinander dargestellt werden.

Ein vertikales (senkrechtes) Menü wird nun durch eine kleine Änderung innerhalb eines @media-Query-Blocks erzeugt:

```
@media screen and (max-width: 935px) {
  nav li {
    display: block;
    margin: 5px;
  }
}
```

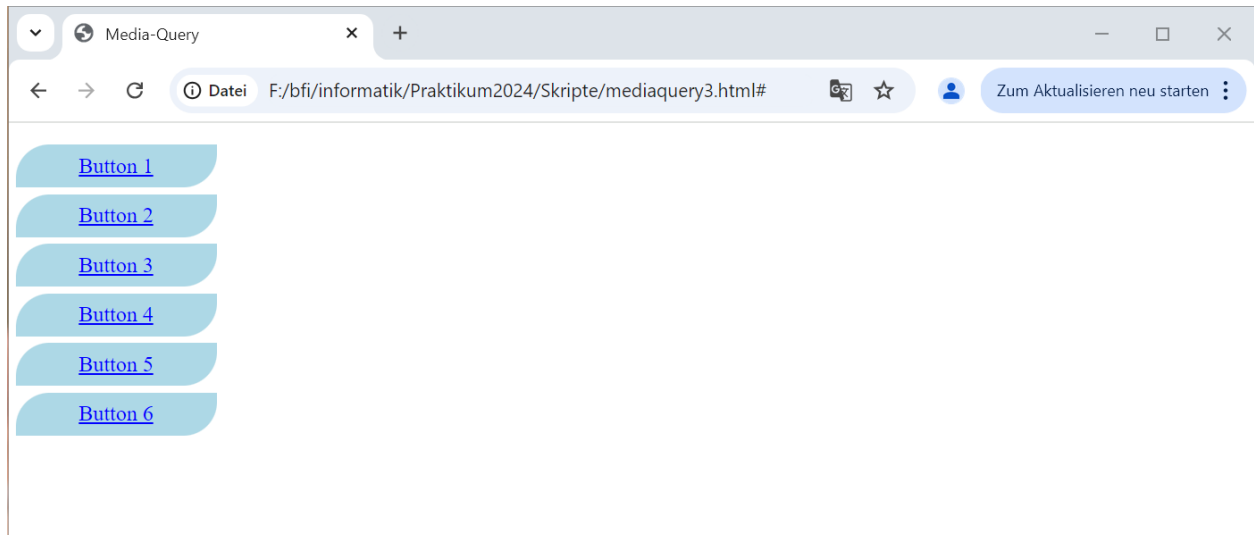


Abbildung 4: Das Menü wird im Beispiel nur dann in vertikaler Form erscheinen, wenn die Breite des Viewports maximal die programmierte Weite in px hat: 935px.

Merke: Ein responsives Webdesign kann mithilfe der @media-Queries erreicht werden. Die Einschränkung auf bestimmte Bildschirmbreiten oder andere Medieneigenschaften alleine machen jedoch noch kein responsives Design aus. Wichtig ist, dass innerhalb der @media-Regelblöcke klare Formate festgelegt werden, die ausschließlich für den jeweiligen Fall zutreffen. Auf gar keinem Fall sollte der gesamte allgemeine CSS-Code in diese Blöcke kopiert und angepasst werden. Der CSS-Code würde sehr rasch enorm groß werden. Für ein responsives Design sei auch auf die nachfolgenden Module hingewiesen.

Wenngleich auch eine eindeutige Breitenangabe mittels „width“ möglich ist, sollte stets beachtet werden, dass Viewport-Breiten variieren. Das liegt nicht nur an den verschiedenen Ausgabegeräten (PC-Monitor, Tablet, Smartphone) und deren verschiedenen Bildschirmauflösungen, sondern auch daran, dass die Besucher einer Webseite das Browserfenster in einer individuellen Größe auf dem Bildschirm platzieren können. Für die Medienabfrage der Viewport-Breite sollte deswegen auf feste Breiten („width“) in der Bedingung verzichtet werden. Besser sind hier Angaben der Mindest- oder Maximalwerte („min-width“ oder „max-width“).

@media-Query mit dem Medienmerkmal „height“ bzw. „min-/max-height“

Die Medienabfrage funktioniert natürlich auch mit der Höhe des Viewports. Die Syntax ist entsprechend ähnlich wie zuvor bei der Viewport-Breite beschrieben.

Experimentieren Sie mit Medienabfragen für den Bildschirm mit verschiedenen Bildschirmhöhen. Probieren Sie auch Kombinationen von Medienabfragen aus, die sowohl Höhen als auch Breiten untersuchen.

@media-Query mit dem Medienmerkmal „orientation“

Es wird immer wieder gefragt, wie eine Webseite für ein ganz bestimmtes Smartphone optimal gestaltet werden kann. Eine echte Antwort auf diese Frage kann es nicht geben. Zwar simulieren verschiedene

Webbrowser die Eigenschaften namhafter Smartphones, was es erlaubt, die eigenen Formate daran zu erproben, doch die Entwicklung schreitet schnell voran und auch die Displays werden immer leistungsfähiger und bieten hohe Auflösungen. Medienabfragen, die sich allein an Höhe und Breite des Viewports orientieren, können nicht immer optimale Ergebnisse liefern.

Es bietet sich aber als zusätzliche Option an, die Ausrichtung eines Smartphone-Displays zu berücksichtigen. In den meisten Fällen wird das Gerät zunächst „hochkant“ benutzt. Das bedeutet, dass die Breite des Viewports schmaler ist als dessen Höhe. Man spricht allgemein auch von der „Portrait“-Ansicht. PC-Monitore sind dagegen vorwiegend in der Breite dominant. Man spricht von der „Landscape“-Ansicht.

Selbstverständlich kann dies nicht allgemein garantiert werden, dann insbesondere bei einer schmalen Fenster-Einstellung stimmen die Klassifizierungen nicht mehr. Trotzdem soll eine Medieneigenschaft in den @media-Queries erwähnt werden, die sich auf die Ausrichtung des Viewports bezieht: „orientation“

Verändern Sie die @media-Query aus dem letzten Beispiel nach dem folgenden Code-Beispiel:

```
@media screen and (orientation: portrait) {  
  nav li {  
    display: block;  
    margin: 5px;  
  }  
}
```

Betrachten Sie nun die Seite mit verschiedenen Breiten des Viewports.



Abbildung 5: Das Verhältnis von Breite zu Höhe ist eindeutig: Das Fenster ist deutlich höher als breit (orientation: portrait). Ein durchaus markantes Merkmal eines Smartphones.

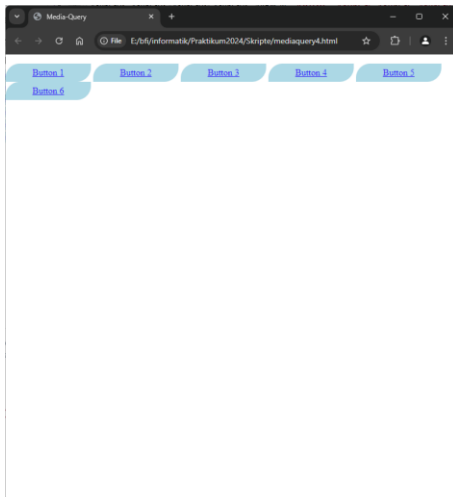


Abbildung 6: Tatsächlich ist dieses Fenster in der Höhe etwas größer als die Breite (wenige Bildpunkte). Trotzdem wird der Portraitmodus nicht erkannt. Das Menü wird weiterhin als horizontale Leiste, jedoch mit einem (unerwünschten) Umbruch dargestellt.

Als Werte für die Medieneigenschaft „orientation“ kommen in Frage:

- **landscape** (Bildschirm ist breiter als hoch – klassischer PC-Monitor)
- **portrait** (Bildschirm ist höher als breit – klassisches Smartphone-Display)

@media-Query mit dem Medienmerkmal „aspect-ratio“

Eine Alternative zur recht rudimentären Medienabfrage mit „orientation“ bietet die Medieneigenschaft „aspect-ratio“. Hierbei handelt es sich um das Verhältnis der Viewport-Breite zur Viewport-Höhe. Dieses Seitenverhältnis hat beispielsweise für TV-Geräte eine wichtige Bedeutung. So wurden ältere „Röhrengeräte“ (CRT = Cathode Ray Tube) mit einem Seitenverhältnis von 4:3 (1,333) gefertigt. Moderne Flachbild-TV-Geräte verwenden dagegen das der Kino-Leinwand ähnlichere Format 16:9 (= 1,778).

Das Seitenverhältnis des Viewports lässt sich mit einer Medienabfrage überprüfen und darauf basierend ein spezielles Format programmieren. Das folgende Beispiel zeigt eine einfache Webseite, deren Hintergrundfarbe in ein dunkles Grün wechselt, wenn das Seitenverhältnis kleiner oder gleich 16:9 (bzw. dezimal: 1,778) ist:

```
@media screen and (aspect-ratio <= 16/9) {
  body {
    background-color: green;
  }
}
```

Achtung: Hier gilt es, die die Schreibweise (Syntax) zu beachten! Hinter der Medieneigenschaft „aspect-ratio“ wird kein Doppelpunkt gesetzt.

Die Abbildung 7 und Abbildung 8 zeigen die gleiche Webseite, wie sie in diesem Modul als Beispiel verwendet wurde. Der Unterschied in den beiden Abbildungen liegt in der Viewport-Breite. Während in Abbildung 7 eine Breite von 1291px eingestellt wurde, ist die Viewport-Breite in Abbildung 8 um einen Bildpunkt kleiner. Die Höhe des Viewports ist hier in beiden Fällen gleich 726px. Entsprechend der Medienabfrage mit dem Medienmerkmal `aspect-ratio <= 16/9` wird der in der @media-Query eingeschlossene Regelblock umgesetzt und das Element `<body>` mit einer grünen Hintergrundfarbe formatiert.

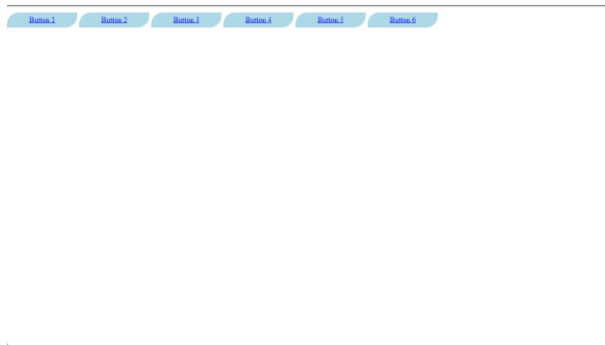


Abbildung 7: Viewport-Abmessungen: 1291px x 726px

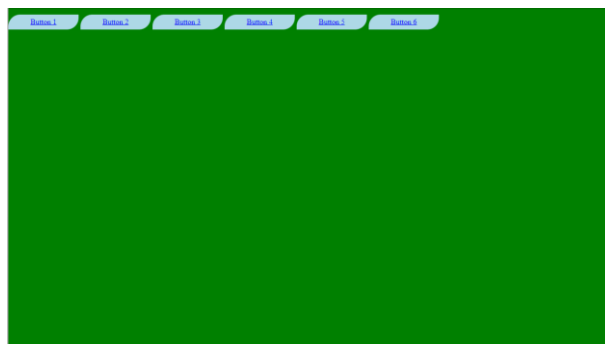


Abbildung 8: Viewport-Abmessungen: 1290px x 726px

Die @media-Queries funktionieren auch mit dezimalen Werten. So kann anstelle des geläufigen Standard-Verhältnisses 16/9 auch der Dezimale Wert 1,788 oder für das Seitenverhältnis 4:3 auch der dezimale Wert 1,333 eingesetzt werden.

Natürlich wäre es viel zu schön, um wahr zu sein, wenn mit einer ganz einfachen Festlegung eines Seitenverhältnisses eine optimale Einstellung für ein Smartphone möglich wäre. Die Seitenverhältnisse gängiger Geräte sind alles andere als identisch. Die Spanne reicht (digitale Werte) von 1,778 bis zu 4,872 (vgl. Abbildung 9). Tendenziell lassen sich aber mit guten Näherungen die Formate für verschiedene Smartphone-Modelle anpassen.

Auflösung	Format	Bezeichnung	Hertz	Beispiel-Gerät
1600 x 720 Pixel	20:9	HD+	60 Hz	Samsung Galaxy A03s
1334 x 750 Pixel	16:9	HD+	60 Hz	iPhone SE 2022
2.340 x 1.080 Pixel	19,5:9	Full-HD+	120 Hz	Samsung Galaxy S22
2.400 x 1.080	20:9	Full-HD+	90 Hz	Pixel 7
2.400 x 1.080 Pixel	20:9	Full-HD+	120 Hz	Samsung Galaxy A53 5G
2.400x1.080 Pixel	20:9	Full-HD+	120 Hz	Honor 70 5G
2.640 x 1.080	22:9	Full-HD+	120 Hz	Samsung Galaxy Z Flip 4*
2532 x 1170 Pixel	19,5:9	XDR	60 Hz	iPhone 14
2796 x 1290 Pixel	19,5:9	XDR	120 Hz	iPhone 14 Pro Max
3120 x 1440 Pixel	19,5:9	Quad-HD+	120 Hz	Pixel 7 Pro
3.088 x 1.440 Pixel	19,3:9	Quad HD+	120 Hz	Samsung Galaxy S22 Ultra
3200x1440 Pixel	20:9	Quad HD+	120 Hz	Xiaomi 12 Pro
3840 x 1644 Pixel	21:9	UHD	120 Hz	Sony Xperia 1 IV

Abbildung 9: Bildformate verschiedener Smartphones - Quelle: [Handyhase.de](https://www.handyhase.de), Zugriff: 21.10.2024

Größenbereiche mit @media-Queries:

In den Ausführungen zu den Medienmerkmalen für die Breite des Viewports (entsprechend auch die Höhe) wurde folgende Formulierung verwendet:

```
@media screen and (max-width: 935px) {...}
```

Es lassen sich aber auch Medienmerkmale so formulieren, dass damit genau definierte Größenbereiche beschrieben werden. Dabei kommen Vergleichsoperatoren zum Einsatz, wie sie auch aus anderen Programmiersprachen für die Formulierung von Bedingungen in „bedingten Verzweigungen“ bereits bekannt sind:

- < ist kleiner als
- <= ist kleiner als oder gleich
- = ist gleich
- >= ist größer als oder gleich
- > ist größer als

Im folgenden Beispiel sollen CSS-Regeln für einen genau in der Breite definierten Bereich formuliert werden, der von einschließlich 800px bis einschließlich 1200px reichen soll:

```
@media screen and (800px <= width <= 1200px) {...}
```

Aufgabe: Experimentieren Sie mit anderen Breiten und auch Höhen!

@container

Die Medienabfragen mit @media beziehen sich auf die Eigenschaften des Viewports. Recht nützlich sind auch vergleichbare Abfragen, die sich auf HTML-Container beziehen. Hier können sowohl Abfragen zur **Größe** (Höhe, Breite) aber auch zu den im Container verwendeten **Stilen**¹ bzw. **Formaten** formuliert werden.

Mit den @container-Abfragen können gezielt Eigenschaften von HTML-Containern ausgewertet und in entsprechende Formatierungen umgesetzt werden. Im Gegensatz zu den @media-Queries sind die @container-Queries nicht auf die Eigenschaften des Viewports und damit nicht direkt an das Anzeigegerät gebunden. Sie beziehen sich also gezielt auf einzelne Teile der Webseite.

Wichtig: @container ist eine noch recht neue Abfrageform, die zwar durchaus spannende Formatierungsmöglichkeiten eröffnet, jedoch nicht unbedingt von allen Webbrowsern unterstützt wird. Es ist also bei der Verwendung von @container noch eine gewisse Vorsicht geboten, wenn die Formate auch auf älteren Geräten funktionieren sollen. Nähere Informationen finden Sie auf der Seite caniuse.com.

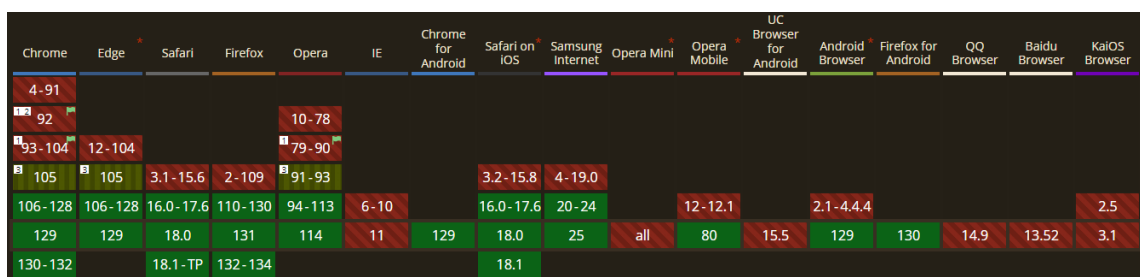


Abbildung 10: Kompatibilität von @container in verschiedenen Webbrowsern - Quelle: caniuse.com, Zugriff: 21.10.2024. Dies bezieht sich jedoch noch nicht auf Stil-Abfragen!

¹ Referenzen auf Stile sind derzeit (Stand: Oktober 2024) noch in der Entwicklungs- bzw. Standardisierungsphase. Dementsprechend sind sie noch nicht wirklich nutzbar. Es wird sich aber für die Zukunft lohnen, die Entwicklung dieser Container-Abfrage zu beobachten.

Container-Typen

Insbesondere für Größen-Container ist es wichtig, zunächst einen **Container-Typ** zu definieren. Dieser Typ kann entweder

- **inline-size** oder
- **size**

sein.

Der Container-Typ „**inline-size**“ lässt Breitenabfragen (width, min-width, max-width) zu. Der Container-Typ „**size**“ erlaubt auch Höhenabfragen.

Wichtig ist, dass die letztlich zu formatierenden Elemente einen „Eltern-Container“ benötigen. Dies kann beispielsweise ein <div>-Element sein. In der Regel werden jedoch statt <div> eher zum Inhalt passende **semantische Elemente** verwendet (siehe dazu auch: Modul 5 – HTML-Body).

Container-Struktur und Container-Name

Es sei eine Webseite nach dem in den vorangegangenen gezeigten Modulen angenommen: Alle <article>-Elemente sind Teil eines Haupt-Containers: das Element <main>.

HTML-Code:

```
<main>
  <article>
...
  </article>
  <article>
...
  </article>
...
</main>
```

Hier bietet es sich an, <main> als Eltern-Container zu definieren.

CSS-Code:

```
main {
  container-type: size;
}
```

@container - Größenabfragen

Der Container soll seine Hintergrundfarbe auf „rot“ verändern, wenn dessen Breite > 500px ist. Dies setzt natürlich voraus, dass sich die Containerbreite in irgendeiner Weise verändern kann. Ist das <main>-Element beispielsweise statisch, also fest auf 499px formatiert, ändert sich die Breite nie und es wird auch keine Reaktion geben, es sei denn, der gesamte Viewport wird so schmal, dass der <main>-Container nicht mehr in voller Breite darstellbar ist.

Ist der Container dagegen mit einer **relativen Breite** formatiert – hier soll als Beispiel eine Breite von 75% angegeben werden – dann hängt die Breite dieses Containers von der Breite des Bezugselements ab (Dies ist <body>, wenn nichts Anderes vorgegeben ist).

Der CSS-Code muss also etwas ergänzt werden:

```
main {
  container-type: size;
  width: 75%
}
```

Zu beachten: In diesem Beispiel wurde der container-type „size“ verwendet. Wie bereits ausgeführt, gibt es auch den Typus „inline-size“.

@container (width >= 500px)

```
{
  article {
    background-color: red;
  }
}
```

Die folgenden Abbildungen zeigen das Ergebnis:

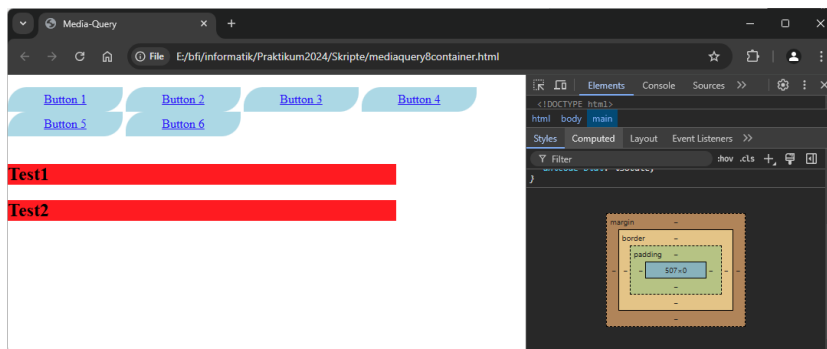


Abbildung 11: Der <main>-Container wird in seiner Breite grundsätzlich 75% der Bildschirmbreite des übergeordneten Elements ausmachen. Damit ist die Breite des Containers nicht statisch an eine bestimmte Bildpunktzahl gebunden. Ist nun die so entstehende Breite größer oder gleich 500px, wird den darin zugewiesenen <article>-Elementen eine rote Hintergrundfarbe zugewiesen.

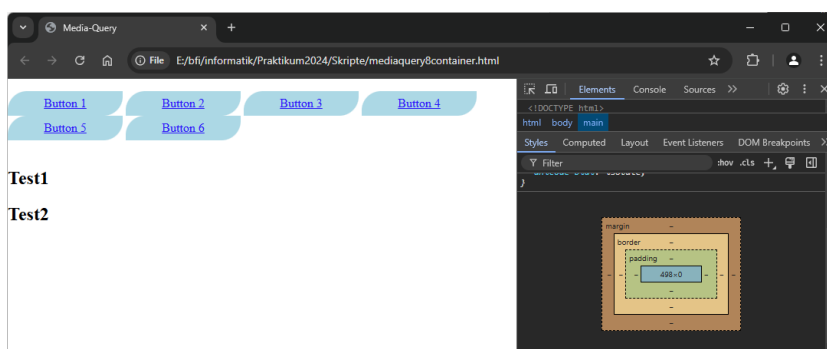


Abbildung 12: Der <main>-Container ist kleiner als 500px - hier: 498px - deswegen ändert sich der Hintergrund nicht.

Natürlich können dem <article>-Element auch Eigenschaften zugewiesen werden, die nicht von der Breitenänderung abhängig sein sollen und grundsätzlich gelten. Diese Formate werden wie bisher außerhalb der Container-Abfrage programmiert:

```
main {
  container-type: size;
  width: 75%
}
```

```

article {
  text-align: center;
}

@container (width >= 500px)
{
  article {
    background-color: red;
  }
}

```

Bitte beachten: Die Textausrichtung wurde mit der CSS-Eigenschaft „text-align: center“ zentriert. Der <main>-Container und damit natürlich auch dessen Inhalte werden aber auf eine Breite von 75% des übergeordneten Elements begrenzt. Deswegen erscheint der Text nicht wirklich auf der Seite zentriert. Die Zentrierung betrifft allein die Abmessungen des <main>-Elements und – über die entsprechende Vererbung der Eigenschaft – auch dessen Inhalte.

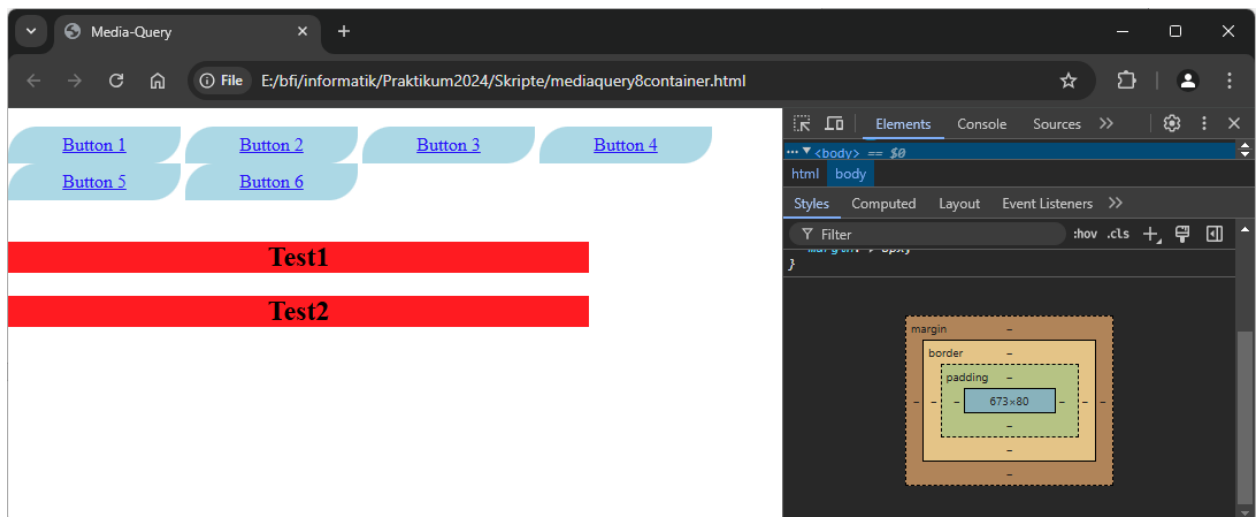


Abbildung 13: Die Zentrierung der Schrift innerhalb der <article>-Elemente erscheint nicht wirklich zentriert. Das liegt daran, dass das <article>-Element die prozentuale Breite vom <main>-Container "erbt".

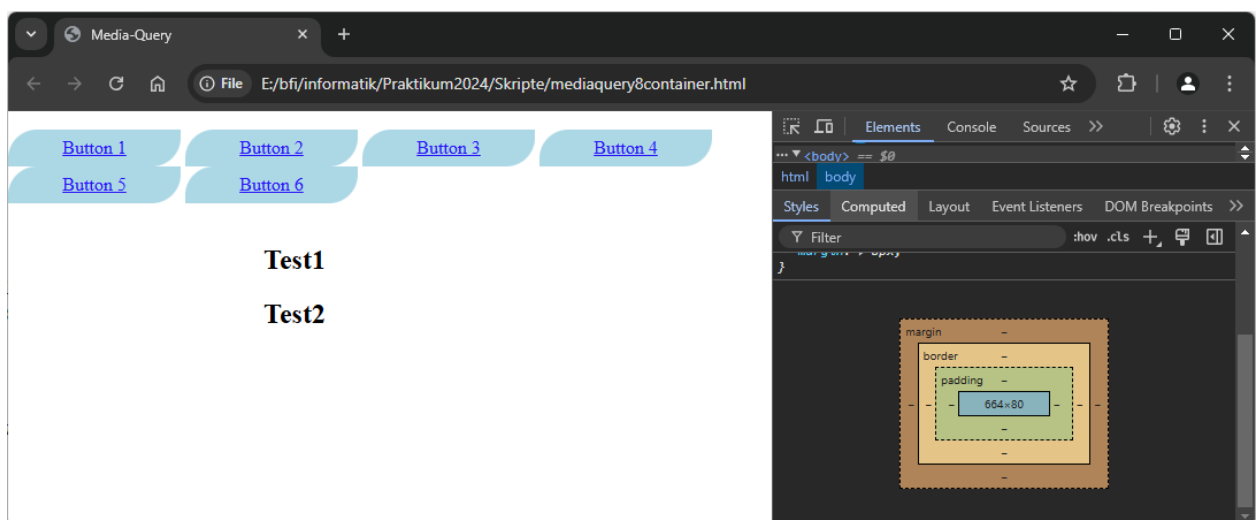


Abbildung 14: Unabhängig von der allgemeinen Formatierung des <article>-Elements bleiben die von der @container-Abfrage abhängigen Formate nach wie vor wirksam.

@container – Stil-Abfragen

In der aktuellen Entwicklung (Stand: Oktober 2024) ist die Abfrage von CSS-Stilen in einem Container zwar angedacht, aber noch nicht zuverlässig umgesetzt. Wie Abbildung 15 zeigt, sind die ersten Ansätze bisweilen noch in sehr wenigen Webbrowsern implementiert. Hier gilt es, die Standardisierungsprozesse des World Wide Web-Consortiums (W3.org) zu beobachten.

Chrome	Edge	Safari	Firefox	Opera	IE	Chrome for Android	Safari on iOS	Samsung Internet	Opera Mini	Opera Mobile	UC Browser for Android	Android Browser	Firefox for Android	QQ Browser	Baidu Browser	KaiOS Browser
4-106	12-106			10-92												
107-110	107-110			93-97				4-21								
111-128	111-128	3.1-17.6	2-130	98-113	6-10		3.2-17.6	22-24		12-12.1		2.1-4.4.4				2.5
129	129	18.0	131	114	11	129	18.0	25	all	80	15.5	129	130	14.9	13.52	3.1
130-132		18.1-TP	132-134				18.1									

Abbildung 15: Bitte keine spektakulären "Universal-Wunder" erwarten! Die Stil-Abfragen sind derzeit noch nicht in allen gängigen Webbrowsern implementiert. Quelle: caniuse.com, Zugriff: 21.10.2024

Aufgaben

- 13.1 Beschreiben Sie den wesentlichen Unterschied zwischen @media und @container!
- 13.2 Formulieren Sie drei @media-Queries, wenn folgende Bedingungen gefordert sind (Achtung: die Reihenfolge in der Aufgabenstellung ist nicht gleich der Reihenfolge der @media-queries):
Format A soll aktiv sein, wenn die Bildschirmbreite von **0px bis 700px** beträgt.
Format B soll aktiv sein, wenn die Bildschirmbreite **mehr als 1920px** beträgt.
Format C soll die **übrigen Bildschirmbreiten** abdecken.
- 13.2 Sehen Sie sich Abbildung 9 an: Wie können Sie @media-Queries kombinieren, damit Sie gute Chancen haben, dass Ihre Webseite die Anzeige auf einem Stand-PC von der eines Handys unterscheiden kann?
- 13.4 Welche grundlegenden **Ausgabemedien** können mit @media-Queries eingestellt werden (Achtung: drei Optionen)?
- 13.5 Was sind die wichtigsten **Medienmerkmale**, die mit @media-Queries ausgewertet werden können?
- 13.6 Sie drucken eine Seite aus, die auf dem Bildschirm mit einem schwarzen Hintergrund und heller Schrift erscheint. Wie erscheint die Seite, wenn sie die Seite auf den Drucker ausgeben?